数据库缓冲管理：CLOCK 时钟置换算法及其与 LRU 的关系

一、核心结论

CLOCK（时钟置换算法），也称 Second-Chance（第二次机会）算法，是一种近似实现 LRU（Least Recently Used，最近最少使用）思想的缓存置换策略。

二者的共同目标是：当缓冲池已满，需要调入新的磁盘页面时，尽量保留近期可能再次访问的页面，优先淘汰较长时间未被访问的页面。

主要区别在于：

- LRU：精确维护页面最近一次使用的先后顺序，淘汰最久未使用的页面。
- CLOCK：通过每个缓存帧的一个引用位（Reference Bit）以及一个循环移动的时钟指针，近似判断页面是否近期被访问。
- 核心思想：引用位为 1 的页面获得一次保留机会，引用位为 0 的页面在被扫描到时成为置换对象。

因此，可以用一句话概括：

> LRU 记录页面使用的先后顺序；CLOCK 通过循环检查引用位，近似维护页面的近期使用信息，从而降低缓存管理开销。

CLOCK 是经典数据库缓冲池置换策略之一。CMU 的数据库系统课程也将其作为 LRU 的近似算法进行介绍。

CMU 15-445/645

二、CLOCK 算法的工作原理

2.1 为什么数据库需要页面置换?

数据库中的数据通常以固定大小的页（Page）为单位存储在磁盘上。

执行 SQL 查询时，数据库管理系统需要将相关数据页从磁盘读入内存中的缓冲池（Buffer Pool）。

假设缓冲池只能容纳 4 个页面：

Buffer Pool（缓冲池）

Frame 0

A

Frame 1

B

Frame 2

C

Frame 3

D

内存中的 4 个缓存帧均已占用

此时，执行一个新的查询，需要访问磁盘页面 E。

由于缓冲池没有空闲帧，数据库必须从 A、B、C、D 中选择一个页面予以淘汰，再将 E 装入对应的缓存帧。

这就产生了缓冲池管理的核心问题：

应该淘汰哪一个页面，才能尽可能减少未来的磁盘 I/O?

LRU 和 CLOCK 都是解决这一问题的策略。

2.2 CLOCK 需要维护哪些信息?

设缓冲池共有 N 个缓存帧，编号为 $0, 1, \dots, N-1$。

对于最基本的 CLOCK 算法，主要维护两个信息。

（1）引用位 Reference Bit

每个缓存帧 i 都有一个引用位：

$R_i \in \{0, 1\}$

其中：

- $R_i = 1$：该页面自上次引用位被清零以来，至少被访问过一次。
- $R_i = 0$：该页面自上次清零以来尚未被再次访问，或者刚装入且初始化为 0。

在下面采用的常见教学版本中，页面被访问或者新装入缓冲池时，将其引用位设置为 1。

特别需要解释的是：引用位并非由时间自动清零，也不会因页面长时间未访问就自动变成 0。 清零操作由时钟指针在置换扫描过程中完成。

（2）时钟指针 Clock Hand

所有缓存帧按照固定顺序组织成逻辑环：

$0 \to 1 \to 2 \to \cdots \to N-1 \to 0.$

用变量 h 表示当前时钟指针指向的缓存帧编号。

每次指针前进时执行：

$h \leftarrow (h + 1) \bmod N.$

也就是说，到达最后一个缓存帧后，指针会重新回到第一个缓存帧。

逻辑上的环形结构并不要求缓存帧在物理内存中排列成圆形。

2.3 CLOCK 的核心规则：第二次机会

当缓冲池已满，需要淘汰某个页面时，CLOCK 从时钟指针当前的位置开始，依次检查缓存帧。

示意图：按 A → B → C → D → A 的顺序循环扫描。蓝色 C 的引用位为 0。

检查每个缓存帧时，分为两种情况。

1

引用位为 1：给予第二次机会

CLOCK 将该引用位清零，并把指针移动到下一个缓存帧。当前页面得以保留。

0

引用位为 0：选择当前页面作为淘汰对象

在页面允许置换的前提下，淘汰当前页面，将新页面装入对应缓存帧，然后继续推进时钟指针。

由此可以理解“第二次机会”的含义：一个页面即使已经成为候选对象，只要引用位为 1，CLOCK 就会暂时保留它。只有指针再次遇到该页面，并发现它的引用位仍为 0 时，它才会被选为牺牲页（Victim Page）。

这实际上是在利用一种粗粒度的时间局部性判断：

$$\boxed{\text{引用位反复被重新设置为 } 1 \implies \text{页面更有机会留在缓冲池}}$$

这里强调“更有机会”，因为 CLOCK 并没有严格记录页面访问时间。

2.4 CLOCK 算法的伪代码

结合数据库缓冲池的实际要求，一个更加完整的 CLOCK 算法如下。

其中，`pin_count` 表示当前有多少使用者正在固定（Pin）该缓存帧。被固定的页面需要保持在缓冲池中，暂时不能参与淘汰。

```
def clock_victim(frames, hand):    """    frames: 缓冲帧数组    hand: 当前时钟指针    返回: (victim_index, new_hand)    """    n = len(frames)    while True:        frame = frames[hand]        # 正在使用的页面不能淘汰        if frame.pin_count > 0:            hand = (hand + 1) % n            continue        if frame.ref == 1:            # 给予第二次机会            frame.ref = 0            hand = (hand + 1) % n        else:            # 找到可以淘汰的页面            victim = hand            hand = (hand + 1) % n            return victim, hand
```

这是用于讲解核心规则的伪代码。实际实现还需要处理所有帧均被固定、并发修改和 I/O 等待等情况。

获得牺牲帧后，缓冲管理器还需要完成旧页面的必要写回、映射关系更新和新页面读入。按照本文的约定，新页面的引用位初始化为 1。

三、示例一：CLOCK 如何一步步选择牺牲页?

假设缓冲池中有 4 个缓存帧，初始状态如下：

缓存帧	页面	引用位
0	A	1
1	B	1
2	C	0
3	D	1

时钟指针目前指向 A，按照 A → B → C → D → A 的顺序循环。

现在，需要访问新页面 E，且 E 不在缓冲池中。

逐步执行 CLOCK

1 / 4

Frame 0

A

R = 1

Frame 1

B

R = 1

Frame 2

C

R = 0

Frame 3

D

R = 1

初始状态

指针指向 A。缓冲池已满，需要装入 E。

上一步下一步

完成上述操作后，缓冲池状态为：

缓存帧	页面	引用位
0	A	0
1	B	0
2	E	1
3	D	1

时钟指针现在指向 D。

继续访问 A，然后访问 F

假设接下来发生以下两次访问。

第一次：访问 A

A 已在缓冲池中，属于缓存命中（Cache Hit）。

因此：

$R_A : 0 \to 1$

时钟指针保持在 D，缓冲池中的页面不发生变化。

第二次：访问 F

F 不在缓冲池中，需要选择牺牲页。

从 D 开始扫描：

1. D 的引用位为 1，清零，指针前进到 A。
2. A 的引用位为 1，清零，指针前进到 B。
3. B 的引用位为 0，淘汰 B，将 F 装入其缓存帧。

最终得到：

页面	引用位
A	0
F	1
E	1
D	0

时钟指针指向 E。

这个例子说明了两个重要特点。

第一，CLOCK 在置换过程中可能检查多个页面，直到发现引用位为 0 的候选页。

第二，页面访问不会直接移动时钟指针。页面访问通常只是将对应的引用位设置为 1，只有置换扫描才会推进指针。

四、CLOCK 与 LRU 有什么本质关系?

这是整个算法最值得深入解释的部分。

4.1 LRU 是如何工作的?

LRU（Least Recently Used）基于时间局部性（Temporal Locality）原理：

> 最近访问过的页面，在不久的将来也可能再次被访问。

假设缓冲池包含 A、B、C 三个页面，访问顺序为：

$A \to B \to C \to A.$

此时，按照最近一次访问时间从远到近排列：

最久未使用（LRU）

最近使用（MRU）

B

最旧

C

中间

A

最新

LRU 使用次序：B → C → A

如果此时要调入 D，LRU 会淘汰 B，因为 B 最久没有被访问。

精确 LRU 可以定义为：

$$v_{\mathrm{LRU}} = \arg\min_{i \in \mathcal{E}} t_i,$$

其中：

- $\mathcal{E}$ 表示当前允许被淘汰的缓存帧集合。
- t_i 表示缓存帧 i 中页面的最近一次访问时间。
- v_{LRU} 表示最终选择的牺牲帧。

在数据库系统中，具体实现可能以页面的最近一次 Pin、Unpin 或缓冲访问事件更新 LRU 信息，取决于采用的缓冲管理接口。

4.2 CLOCK 为什么可以看作 LRU 的近似?

LRU 希望识别的是：

哪个页面最久没有被使用?

CLOCK 实际掌握的是：

$\text{哪个页面自引用位上次清零以来尚未重新访问?}$

二者存在关联：

如果一个页面经常被访问，那么它的引用位就更容易被重新设置为 1。因此当时钟指针扫描到该页面时，它较有可能获得第二次机会。

另一方面，如果某个页面长时间没有被访问，那么它的引用位一旦被清零，就会一直保持为 0，直到下次访问或被淘汰。

因此，在一定的访问模式下，CLOCK 能够近似区分相对活跃与不活跃的页面。

但二者存在一个关键区别：

引用位只有 1 bit，无法记录页面访问的确切时间及访问顺序。

例如：

- A 在 1 秒前被访问。
- B 在 10 秒前被访问。
- C 在 100 秒前被访问。

假设三者的引用位均为 1，CLOCK 无法根据这三个引用位判断哪个页面访问得更早。

LRU 则可以准确判断 C 是最久未访问的页面。

这就引出一个容易让学生产生疑问的现象。

4.3 一个反例：CLOCK 可能淘汰刚刚访问过的页面

考虑容量为 3 的缓冲池，初始为空，页面访问序列为：

$$A,\ B,\ C,\ A,\ D,\ A.$$

我们约定：新装入页面引用位设置为 1，装入后指针指向下一个缓存帧。

步骤	访问页面	CLOCK 缓冲帧状态（含引用位）	指针位置
1	A	A(1), –, –	1
2	B	A(1), B(1), –	2
3	C	A(1), B(1), C(1)	0
4	A	A(1), B(1), C(1)	0
5	D	D(1), B(0), C(0)	1
6	A	D(1), A(1), C(0)	2

重点分析第 5 步。

访问 A、B、C、A 后，三个页面的引用位全部为 1，指针指向 A。

现在发生 D 的缺页：

- 第一次检查 A，发现 $R_A = 1$，清零。
- 检查 B，发现 $R_B = 1$，清零。
- 检查 C，发现 $R_C = 1$，清零。
- 指针回到 A，此时 $R_A = 0$，因此淘汰 A。

尽管 A 是刚刚访问过的页面，CLOCK 仍然可能将其淘汰！

如果使用 LRU，在访问 D 时则应淘汰 B。

CLOCK

第 5 步后

D

B

C

淘汰 A

LRU

第 5 步后

A

C

D

淘汰 B

第 6 步再次访问 A 时，CLOCK 发生缓存未命中，而 LRU 发生缓存命中。

这说明：

$\boxed{v_{\text{CLOCK}} \neq v_{\text{LRU}} \quad \text{在一般情况下是可能的。}}$

CLOCK 对 LRU 的近似是启发式的，不保证每次选择与 LRU 一致，也不保证在所有访问序列下获得相同的命中率。

4.4 二者的性能与实现开销比较

比较维度	LRU	CLOCK
决策依据	最近一次访问的时间顺序	引用位与循环扫描位置
访问时更新	更新时间戳或链表位置	通常只设置引用位
元数据	时间信息或链表指针	每帧一个引用位，另有指针
查找牺牲页	精确 LRU 链表通常为 $O(1)$	最坏 $O(N)$
并发管理开销	频繁维护共享使用顺序	访问路径上的维护通常较轻
置换精度	精确遵守 LRU 顺序	对 LRU 的近似
多次访问的区分	能识别最后访问时间	基本 CLOCK 无法区分 1 次与多次访问

这里需要强调两点。

首先，CLOCK 并不能保证每次置换都比 LRU 快。它可能需要扫描多个缓存帧，最坏情况下其牺牲页查找复杂度为 $O(N)$。

其次，精确 LRU 通过哈希表和双向链表同样可以做到每次命中和置换的 $O(1)$ 时间复杂度。但每次访问通常需要修改链表结构，在高度并发的数据库系统中，这可能增加同步与缓存一致性开销。

因此，CLOCK 的主要优势体现在更简单、通常更低成本的访问元数据维护上，而不意味着每个操作都有更低的最坏时间复杂度。

五、数据库缓冲管理器中还需要考虑什么？

操作系统的页面置换与数据库缓冲池置换思想相通，但数据库缓冲管理器还要处理多个重要约束。

5.1 固定页（Pinned Page）不能被淘汰

数据库执行一个查询时，可能正在使用某个缓冲页。为了确保执行过程中该页面不会被其他操作置换，需要将其固定。

定义：

$$P_i = \mathrm{pinCount}(i).$$

其中，P_i 表示缓存帧 i 当前的固定计数。

只允许从满足

$$P_i = 0$$

的缓存帧中选择牺牲页。

因此，CLOCK 扫描一个页面时：

- 若 $P_i > 0$，跳过该缓存帧。
- 若 $P_i = 0$ 且 $R_i = 1$，清零并继续扫描。
- 若 $P_i = 0$ 且 $R_i = 0$，可以选择该帧作为牺牲帧。

这与第二次机会是两个独立条件。固定页的不可淘汰性是一项正确性约束，引用位用于优化置换决策。

当所有缓存帧均处于固定状态时，缓冲管理器需要根据系统设计等待资源释放或报告无法获得可用帧。

5.2 脏页（Dirty Page）需要写回

如果一个页面在装入内存后经过修改，那么内存版本与磁盘版本可能不同，此时它被称为脏页（Dirty Page）。

用 $D_i \in \{0, 1\}$ 表示脏页标志。

当 CLOCK 选择一个牺牲页后：

- $D_i = 0$：一般可以直接回收缓存帧。
- $D_i = 1$：必须保证修改正确持久化，通常需要写回脏页后才能安全复用缓存帧。

如果数据库采用预写式日志（Write-Ahead Logging，WAL），还需要遵循相应的日志持久化顺序要求。

脏页标志决定置换后的 I/O 处理，引用位决定 CLOCK 的候选选择。 基础 CLOCK 通常不直接利用脏页标志决定淘汰顺序。

5.3 为什么真实数据库往往使用 CLOCK 的改进版本?

一个实际例子是 PostgreSQL 的 CLOCK-sweep 策略。

PostgreSQL 使用 `usage_count` 替代简单的单比特引用位，其上限在当前源码中定义为 5。

GitHub+1

基本思想是：

- 页面被使用时，`usage_count` 可以增加，直至上限。
- 时钟指针扫描时，若计数大于 0，则递减并继续。
- 计数为 0 且页面未固定时，该页面可以作为牺牲页。

这使得一个多次被使用的页面可能获得更多次保留机会。

例如，对两个未固定的页面：

$$c_A = 3, \qquad c_B = 1$$

其中 c_A, c_B 为各自的使用计数。

如果二者后续都没有被再次访问，A 在被时钟指针反复检查时，需要经历更多次计数递减才会成为可置换对象。

该机制兼顾一定的访问频率信息，但仍不能精确恢复 LRU 的完整访问顺序。PostgreSQL 源码也明确将计数上限视为置换精度与扫描成本之间的折中。

GitHub

此外，LRU 与基础 CLOCK 都可能遇到顺序扫描污染（Sequential Flooding）问题：扫描一个很大的表时，大量只访问一次的页面进入缓冲池，挤出原本经常使用的页面。

PostgreSQL 对大型顺序扫描等操作采用专门的缓冲环（Buffer Ring）策略，限制这些访问对常规缓冲池的影响。

GitHub

六、适合课堂讲解的三个思考题

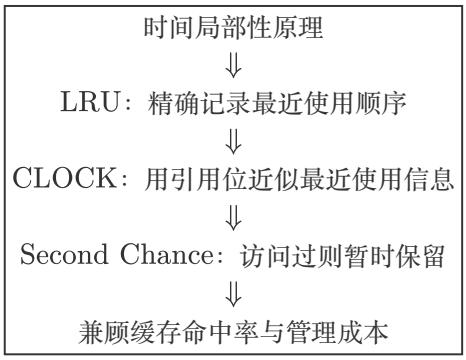
1.如果所有页面的引用位都是 1，CLOCK 会怎样?

2.一个页面被连续访问 100 次，会比只访问 1 次的页面获得更多第二次机会吗?

3.CLOCK 是不是始终比 LRU 差?

七、总结与教学建议

可以用以下逻辑帮助学生建立完整的认识：



建议课堂上重点强调三个理解层次：

1. 机制层：引用位为 1 时清零并跳过，引用位为 0 且页面未固定时选择淘汰。指针按照固定环形顺序推进。
2. 原理层：CLOCK 通过记录页面是否在某个扫描间隔内被访问，粗粒度地近似 LRU 的时间局部性判断。
3. 系统层：数据库还必须处理固定页、脏页写回和并发访问。真实系统往往使用更复杂的计数或缓冲访问策略。

最关键的教学结论是：CLOCK 近似的是 LRU 的决策目标，并不保证复现 LRU 的置换结果。 前文中 CLOCK 淘汰刚访问过的 A、而 LRU 淘汰 B 的反例，特别适合用于检验学生是否真正理解两者的差异。

参考材料

- Carnegie Mellon University, Andy Pavlo, 15-445/645 Database Systems: Memory & Disk Management, Fall 2024，重点参见第 28–39 页的 LRU、CLOCK 与 Sequential Flooding 内容。课程讲义 PDF
- PostgreSQL Global Development Group, Buffer Manager Source Code, `src/backend/storage/buffer/freelist.c`。官方源码
- PostgreSQL Global Development Group, Buffer Manager README，包括 Clock-Sweep 和 Buffer Ring 的设计说明。设计文档